

Answer Key — Inheritance

Instructor reference for the review worksheet — page 1 of 2

part 1 What Is Inheritance? — Q1, Q2

1. `base` • `derived`
2. `is`
3. `inherits` / `derives`

Q2: Circle **all four** — `make`, `model`, `topSpeed`, and `describe()` all become part of every `Car` object through inheritance, regardless of their access level. Access level only controls *who's allowed to reach them*, not whether they exist — that distinction is the whole point of Part 2.

part 2 Access Control in a Hierarchy — Q3

SPECIFIER	SAME CLASS	DERIVED CLASS	OUTSIDE (MAIN())
<code>private</code>	Y	N	N
<code>protected</code>	Y	Y	N
<code>public</code>	Y	Y	Y

Line X does not compile. `make` has no access label in `Vehicle`, so it defaults to `private` — and `private` members are never visible to a derived class, only to the base class itself. One-word fix: change `make` to `protected`.

part 3 Constructor Chaining — Q4, Q5

Q4: Prints **A B C** — construction always runs base-first, all the way down the chain, before the most-derived constructor body finishes. Destructors run in the exact opposite order: **~C ~B ~A**.

Q5 model answer:

```
Motorcycle::Motorcycle(string mk, string mo, int spd, bool sidecar)
    : Vehicle(mk, mo, spd), hasSidecar(sidecar) {}
```

part 4 Overriding vs. Overloading — Q6, Q7

Q6: `v.describe();` prints **"A vehicle."** — `c.describe();` prints **"A car."**. To call the base version from inside the override: `Vehicle::describe();`

1. `Car(string)` / `Car(string, int)` — **OVERLOADED** (same name, different parameter lists, same class)
2. `Vehicle::describe()` / `Car::describe()` — **OVERRIDDEN** (identical signature, redefined in the derived class)
3. `Vehicle::describe()` / `Car::honk()` — **NEITHER** (different names — unrelated functions)
4. `operator+(Fraction)` / `operator+(int)` — **OVERLOADED**

Answer Key — Inheritance

Instructor reference for the review worksheet — page 2 of 2

part 5 Hierarchical Inheritance — Q8 (model answer)

```
class Motorcycle : public Vehicle {
    bool hasSidecar;
public:
    Motorcycle(string mk, string mo, int spd, bool sidecar)
        : Vehicle(mk, mo, spd), hasSidecar(sidecar) {}

    void describe() {
        Vehicle::describe();
        cout << (hasSidecar ? "Has a sidecar.\n" : "No sidecar.\n");
    }
};
```

part 6 Find the Error — Case A, B, C

Case A `class Car : Vehicle` with no keyword defaults to **private** inheritance (mirrors W01: no label = private). That turns every inherited `public` member of `Vehicle`, including `describe()`, into a `private` member of `Car`. Line X (`c.describe();` in `main()`) does **NOT compile** — `describe` is private within this context. Fix: `class Car : public Vehicle`.

Case B Does **NOT compile**. `Car(int doors)` never chains to a `Vehicle` constructor, so the compiler tries to call `Vehicle()` implicitly — but `Vehicle` only has `Vehicle(string mk)`, no default constructor. Missing: a way to supply `mk`. Fix: give `Car` a `string` parameter too and chain it — `Car(string mk, int doors) : Vehicle(mk), numDoors(doors) {}`.

Case C Does **NOT compile**. `protected` only widens access for **derived classes** — it does nothing for outside code. `main()` is not a member of `Vehicle` or any class derived from it, so `v.make` is just as unreachable as if `make` were `private`.

part 7 A Peek Ahead: Polymorphism

No graded response here — this section previews `virtual` functions for next time, the same way Worksheet 04 previewed shallow/deep copying before Worksheet 03 covered it in full. If a student asks: yes, marking `Vehicle::describe()` as `virtual` would make `v→describe();` print "A car." instead — that's the fix, and it's the entire subject of the next lesson.